Hill Climbing Search

También conocido como *Greedy Local Search*.

- Comienza con una solución inicial aleatoria.
- Luego, se encarga de *encontrar en vecindad, una mejor solución*.
- El algoritmo terminal cuando *no encuentra una mejor solución* ó, acabado el *número de iteraciones*.

> **Indagación de espacio de solución**:
> La interpretación computacional *de cómo manejamos la solución* es importante. Por ejemplo, la solución se puede tratar como una lista de 0's o 1's (problema de KnapsackProblem).
> Por lo tanto, para cada problema o ejemplo, la indagación computacional de vecinos, no es más que permutaciones de ese espacio de solución **sin salir de la vecindad**.
> Un vecino válido para un espacio de soluciones "pivote", sería por ejemplo un cambio breve de algún elemento de ese arreglo.
>
> **Saltando de vecindad**:
> A medida que cambiamos el espacio de solución `s` mediante alguna lógica de estas heurísticas, los van a aparecer nuevos vecinos. Y eso permite la posibilidad de encontrar cada vez mejores vecinos.

Simple Hill Climbing

Se tiende a *estancar en los óptimos locales*.

***Simple hill-climbing* – Esquema general**

```
algoritmo simple_hill_climbing(F, c)
      t := 0
      s := seleccionar solución inicial en F
      repetir
            r := greedy_tweak(copy(s))
            si (r <> vacia) entonces
                  s := r
            fsi
            t := t + 1
      hasta (r = vacia) o (t = MAX)
      retornar s
fin
```

Consiste en iterar por los vecinos y encontrar alguno mejor.

- *greedy_tweak*: Consiste *indagar* por una solución vecina diferente de manera aleatoria.

Stochastic Hill Climbing

Tiene la ventaja que tiene menos cómputo por las iteraciones.
Pero *tarda más en converger*.

***Stochastic hill-climbing* – Esquema general**

```
algoritmo stochastic_hill_climbing(F, c)
     t := 0
     s := seleccionar solución inicial en F
     repetir
          r := random_tweak(copy(s))
          si (c(r) es mejor que c(s)) entonces
               s := r
          fsi
          t := t + 1
     hasta (s es una solución ideal) o (t = MAX)
     retornar s
fin
```

Es más sencillo. En vez de indagar vecinos; selecciona uno de manera aleatoria.

Steepest Ascent Hill Climbing

La búsqueda competitiva

```
algoritmo steepest_ascent_hill_climbing(F, c)
    t := 0
    n := número de mejoras deseadas
    s := seleccionar solución inicial en F
    repetir
        r := tweak(copy(s))
        para i := n - 1 decrementando hasta 0 hacer
            w := tweak(copy(s))
            si (c(w) es mejor que c(r)) entonces
                r := w
            fsi
        fpara
        si (c(r) es mejor que c(s)) entonces
            s := r
        fsi
        t := t + 1
    hasta (s es una solución ideal) o (t = MAX)
    retornar s
fin
```

Es un poco más complejito.

- Tomando en cuenta el vecino `r`, genera otro vecino a partir de `s` llamado `w`.
- `r` y `w` van a estar compitiendo para ver quien es mejor, solo que `w` tiene más oportunidades de ganar porque tiene una ventana de indagación `n`, que es un parámetro del algoritmo.
- `n` indica cuantas indagaciones puede hacer `w`.
- Finalmente, la idea es reemplazar `r` con `w`, para así compararlo con `s`.
- La idea es que en vez de tomar uno vecino aleatorio, comparar que vecino es el mejor para encontrar una nueva vecindad.

Random Resstart Hill Climbing

No tengo paciencia para buscar

Tiene la ventaja que *evita quedarse varado en óptimos locales*. Es su principal propósito.
Es más *lento*.

```
algoritmo Random_restart_hill_climbing(F, c)
    t := 0
    T := distribución de posibles intervalos de tiempo
    s := seleccionar solución inicial en F
    best := copy(s)
    repetir
        time := seleccionar tiempo aleatorio en T
        repetir
            r := tweak(copy(s))
            si (c(r) es mejor que c(s)) entonces
                s := r
            fsi
            time := time - 1
        hasta (s es una solución ideal) o (time = 0)
        si (c(s) es mejor que c(best)) entonces
            best := s
        fsi
        t := t + 1
        s := seleccionar solución aleatoria en F
    hasta (best es una solución ideal) o (t = MAX)
    retornar best
fin
```

Consiste en reiniciar la solución inicial.

- Existe un tiempo `time` (que no es más que un simple contador), que dicta el momento de un reinicio de *solución pivote* o *solución inicial*.
- Esto fuerza a que en vez de indagar vecinos, salte aleatoriamente a otra vecindad sin olvidar la mejor solución `best`.
- `time` también se puede interpretar como las iteraciones necesarias para buscar un vecino mejor dentro de un espacio de solución `s` (un mini Simple Hill Climbing de toda la vida).
- Siempre va a saltar; pero no siempre va a cambiar el mejor (*Memoria a largo plazo*).